基于 Pytorch 的深度学习实践 (五)

YYFL

March 2026

1 优化算法原理

优化算法的作用是通过不断更新模型参数，使损失函数逐渐减小，从而提高模型的预测能力。在深度学习中，训练的本质就是通过优化算法找到一组较优的参数，使模型在训练数据上表现更好。

1.1 梯度下降的基本思想

设模型参数为 θ，损失函数为 $L(\theta)$，则最基本的梯度下降更新公式为：

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)$$

其中：

- θ_t 表示第 t 次迭代时的参数；
- η 表示学习率（learning rate）；
- $\nabla L(\theta_t)$ 表示损失函数对参数的梯度。

梯度表示损失函数上升最快的方向，因此参数更新时要沿着梯度的反方向移动，从而使损失减小。

1.2 梯度下降的三种形式

根据每次更新时所使用的数据量不同，梯度下降可以分为以下三种形式。

1.2.1 批量梯度下降（Batch Gradient Descent）

批量梯度下降在每次更新参数时，使用整个训练集来计算梯度：

$$\theta_{t+1} = \theta_t - \eta \frac{1}{N} \sum_{i=1}^{N} \nabla L(\theta_t; x_i, y_i)$$

其中 N 表示训练集样本总数。

特点：

- 梯度计算准确，更新方向稳定；
- 计算量较大，训练速度较慢；
- 不适合大规模数据集。

1.2.2 随机梯度下降（Stochastic Gradient Descent, SGD）

随机梯度下降每次只使用一个样本来更新参数：

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t; x_i, y_i)$$

特点：

- 每次更新速度快；
- 梯度波动较大，训练过程不够稳定；
- 有时有助于跳出局部最优。

1.2.3 小批量梯度下降（Mini-batch Gradient Descent）

小批量梯度下降每次使用一小部分样本来计算梯度：

$$\theta_{t+1} = \theta_t - \eta \frac{1}{B} \sum_{i=1}^{B} \nabla L(\theta_t; x_i, y_i)$$

其中 B 表示 batch size。

特点：

- 兼顾计算效率和训练稳定性；
- 适合 GPU 并行计算；
- 是深度学习中最常用的训练方式。

1.3 学习率（Learning Rate）

学习率决定了每次参数更新的步长大小。

- 学习率过大：可能导致损失震荡，甚至无法收敛；
- 学习率过小：训练速度很慢，容易陷入局部区域。

因此，选择合适的学习率对模型训练非常重要。

1.4 Momentum（动量法）

普通 SGD 在训练过程中容易出现震荡，尤其是在某些方向梯度变化较大的情况下。Momentum 方法通过引入“历史梯度”的思想，使参数更新更平滑。

其更新公式为：

$$v_t = \beta v_{t-1} + (1 - \beta) \nabla L(\theta_t)$$

$$\theta_{t+1} = \theta_t - \eta v_t$$

其中：

- v_t 表示动量项；
- β 表示动量系数，通常取 0.9。

优点：

- 可以减少震荡；
- 能加快收敛速度；
- 在深层网络训练中效果较好。

1.5 AdaGrad

AdaGrad 是一种自适应学习率优化算法。它会根据历史梯度信息，自动调整每个参数的学习率。

公式如下：

$$G_t = G_{t-1} + (\nabla L(\theta_t))^2$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t} + \epsilon} \nabla L(\theta_t)$$

其中 ϵ 是一个很小的常数，用于防止分母为零。

优点：

- 能对不同参数使用不同学习率；
- 对稀疏特征问题表现较好。

缺点：

- 学习率会不断减小；
- 后期训练可能变得很慢。

1.6 RMSprop

RMSprop 是对 AdaGrad 的改进。它不累积全部历史梯度平方，而是采用指数加权平均的方法：

$$G_t = \beta G_{t-1} + (1 - \beta) (\nabla L(\theta_t))^2$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t} + \epsilon} \nabla L(\theta_t)$$

优点：

- 避免 AdaGrad 学习率过快减小的问题；
- 训练过程更加稳定；
- 常用于循环神经网络等任务。

1.7 Adam

Adam（Adaptive Moment Estimation）结合了 Momentum 和 RMSprop 的思想，同时考虑梯度的一阶矩和二阶矩信息，是目前最常用的优化算法之一。

首先计算一阶矩和二阶矩：

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\nabla L(\theta_t)$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)\left(\nabla L(\theta_t)\right)^2$$

为了减小初始偏差，还需要做偏差修正：

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

最终参数更新公式为：

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

优点：

- 收敛速度快；
- 对学习率不太敏感；
- 在很多深度学习任务中表现较好。

缺点：

- 有时泛化能力不如 SGD；
- 在部分任务中可能收敛到次优解。

1.8 几种优化算法的比较

优化算法	优点	缺点
BGD	更新稳定	计算量大
SGD	更新快	震荡明显
Momentum	收敛更快，减少震荡	需要调参数
AdaGrad	自适应学习率	学习率后期过小
RMSprop	训练更稳定	仍需调学习率
Adam	收敛快，使用方便	泛化有时不如 SGD

1.9 总结

优化算法决定了模型参数如何更新，是深度学习训练的重要组成部分。从最基础的梯度下降，到 Momentum、AdaGrad、RMSprop 和 Adam，各种优化算法都是在解决训练速度、稳定性和收敛效果等问题。

总体来说：

- SGD 结构简单，但训练波动较大；
- Momentum 能加快收敛并减少震荡；
- AdaGrad、RMSprop 和 Adam 属于自适应学习率方法；
- Adam 是目前实际应用中最常见的优化算法之一。

2 正则与泛化性理论

在机器学习与深度学习中，模型不仅要在训练集上表现良好，还要在未见过的测试数据上保持较好的效果。这种模型对新样本的适应能力称为**泛化能力（Generalization）**。

如果模型在训练集上误差很小，但在测试集上误差很大，说明模型出现了**过拟合（Overfitting）**；如果模型在训练集和测试集上表现都不好，则说明模型可能出现了**欠拟合（Underfitting）**。

为了提高模型的泛化能力，通常需要引入**正则化（Regularization）**方法，对模型复杂度进行约束。

2.1 泛化的基本概念

模型训练的目标不仅是减小训练误差，还要减小测试误差。通常有以下几个概念：

- **训练误差（Training Error）**：模型在训练集上的误差；
- **测试误差（Test Error）**：模型在测试集上的误差；
- **泛化误差（Generalization Error）**：模型在未知数据上的真实误差。

理想情况下，训练误差和测试误差都较小，说明模型学到了数据中真正有用的规律。

2.2 欠拟合与过拟合

2.2.1 欠拟合（Underfitting）

欠拟合指模型过于简单，无法充分学习数据中的规律。其表现为：

- 训练误差较大；
- 测试误差也较大；
- 模型表达能力不足。

产生欠拟合的常见原因包括：

- 模型过于简单；
- 训练轮数不足；
- 特征提取不充分。

2.2.2 过拟合（Overfitting）

过拟合指模型在训练集上学得过于“仔细”，不仅学习了数据中的真实规律，还学习了噪声和偶然性。其表现为：

- 训练误差很小；
- 测试误差较大；
- 模型在新数据上的表现较差。

产生过拟合的常见原因包括：

- 模型过于复杂；
- 参数过多；
- 训练数据过少；
- 训练时间过长。

2.3 正则化的基本思想

正则化的核心思想是：在损失函数中加入额外的约束项，限制模型参数过大或模型过于复杂，从而提高泛化能力。

一般形式可写为：

$$L_{\text{total}} = L_{\text{data}} + \lambda \Omega(\theta)$$

其中：

- L_{data} 表示原始任务损失；
- $\Omega(\theta)$ 表示正则项；
- λ 表示正则化系数，用来控制正则化强度。

通过加入正则项，模型在拟合训练数据时会受到约束，从而减少过拟合现象。

2.4 L1 正则化

L1 正则化是指在损失函数中加入参数绝对值之和：

$$\Omega(\theta) = \sum_i |\theta_i|$$

因此总损失为：

$$L_{\text{total}} = L_{\text{data}} + \lambda \sum_i |\theta_i|$$

特点：

- 可以使部分参数变为 0；
- 能产生稀疏解；
- 有助于特征选择。

因此，L1 正则化常用于希望模型更加稀疏的场景。

2.5 L2 正则化

L2 正则化是指在损失函数中加入参数平方和：

$$\Omega(\theta) = \sum_i \theta_i^2$$

因此总损失为：

$$L_{\text{total}} = L_{\text{data}} + \lambda \sum_i \theta_i^2$$

特点：

- 不会轻易把参数压到 0；
- 会使参数整体变小；
- 能提升模型的稳定性和泛化能力。

在深度学习中，L2 正则化非常常见，也常被称为**权重衰减（Weight Decay）**。

2.6 Dropout

Dropout 是深度学习中一种常用的正则化方法。它的基本思想是在训练过程中，以一定概率随机“丢弃”一部分神经元，使模型不能过度依赖某些局部特征。

设某层神经元输出为 h，经过 Dropout 后可以表示为：

$$\tilde{h} = m \odot h$$

其中：

- m 是随机采样得到的掩码向量；
- $\odot$ 表示逐元素相乘。

作用：

- 减少神经元之间的共适应；
- 提高模型的鲁棒性；
- 降低过拟合风险。

说明：

- 训练时使用 Dropout；
- 测试时关闭 Dropout，并使用完整网络进行预测。

2.7 Early Stopping（提前停止）

Early Stopping 也是一种常见的正则化策略。其思想是：在训练过程中监控验证集误差，当验证集性能不再提升时，提前停止训练。

优点：

- 简单有效；
- 可以防止模型因训练过久而过拟合；
- 在实际训练中应用非常广泛。

2.8 数据增强（Data Augmentation）

数据增强是通过对训练数据进行变换，构造更多样的样本，从而提高模型泛化能力的方法。

在图像任务中，常见的数据增强包括：

- 随机裁剪；
- 随机翻转；
- 旋转；
- 颜色扰动。

在文本任务中，也可以通过同义替换、随机删除等方式进行简单增强。

作用：

- 扩充训练数据；
- 提高模型对输入变化的适应能力；
- 减少过拟合。

2.9 Batch Normalization 与泛化

Batch Normalization（批归一化）原本主要用于加速训练和稳定梯度，但在一定程度上也具有正则化效果。

其原因在于：每个 batch 的统计量不同，会给模型训练带来一定扰动，从而起到类似噪声注入的作用。

作用：

- 加快模型收敛；
- 稳定训练过程；
- 在一定程度上改善泛化性能。

2.10 偏差与方差

泛化理论中常用**偏差（Bias）**和**方差（Variance）**来分析模型表现。

- **高偏差**：模型过于简单，容易欠拟合；
- **高方差**：模型过于复杂，容易过拟合。

因此，模型训练的一个重要目标，就是在偏差和方差之间找到平衡。

2.11 模型复杂度与泛化能力

一般来说，模型越复杂，表达能力越强，但也越容易过拟合；模型越简单，虽然不容易过拟合，但可能无法充分学习数据规律。

因此，提高泛化能力的关键在于：

- 选择合适的模型复杂度；
- 使用有效的正则化方法；
- 保证训练数据足够丰富；
- 在训练过程中合理控制学习过程。

2.12 几种常见正则化方法比较

方法	主要作用	特点
L1 正则化	约束参数绝对值	产生稀疏解
L2 正则化	约束参数大小	提高稳定性
Dropout	随机丢弃神经元	减少过拟合
Early Stopping	提前终止训练	简单有效
数据增强	扩充训练样本	提高泛化能力

2.13 总结

正则化与泛化性理论的核心目标，是使模型不仅能在训练集上学得好，还能在未知数据上保持良好性能。

总体来说：

- 欠拟合说明模型过于简单；
- 过拟合说明模型过于复杂；

- 正则化通过限制模型复杂度来提高泛化能力；
- L1、L2、Dropout、Early Stopping 和数据增强都是常见方法。

因此，在深度学习中，训练一个好模型不仅要关注优化算法，还要关注模型的泛化能力。